

Práctica del Laboratorio 2

Arquitectura del Computador

30 de Julio de 2026

1. ¿Qué diferencias existen entre registros temporales (\$t0–\$t9) y registros guardados (\$s0–\$s7) y cómo se aplicó esta distinción en la práctica?

Los registros temporales (\$t0–\$t9) no mantienen su valor entre llamadas a funciones, por lo que si llamas a otra función no puedes confiar en que sigan igual. Los registros guardados (\$s0–\$s7) sí mantienen su valor, pero si una función los usa debe guardarlos y restaurarlos.

En la implementación del Quick Sort usamos registros guardados (\$s1–\$s4) porque es recursivo y necesitamos que valores como los índices i y j no se pierdan entre llamadas. En el Merge Sort usamos registros guardados (\$s4, \$s5, \$s7) para mantener los punteros a los arreglos principal y auxiliar durante todo el proceso.

2. ¿Qué diferencias existen entre los registros \$a0–\$a3, \$v0–\$v1, \$ra y cómo se aplicó esta distinción en la práctica?

Uso de registros \$a0–\$a3, \$v0–\$v1 y \$ra

- **\$a0–\$a3 (argumentos):** Sirven para pasar parámetros a las funciones. En Quick Sort y Merge Sort, \$a0 siempre tiene la dirección del arreglo, \$a1 y \$a2 los índices o punteros de inicio y fin.
- **\$v0–\$v1 (retorno):** Devuelven resultados. En nuestros códigos los usamos solo para syscalls (leer/imprimir), no para devolver valores entre funciones porque trabajamos con punteros.
- **\$ra (retorno):** Guarda la dirección a donde volver después de un jal. Es fundamental en las llamadas recursivas de ambos algoritmos.

En el Quick Sort:

- \$a0 = dirección del arreglo

- \$a1 = índice izquierdo
- \$a2 = índice derecho
- \$ra se guarda en la pila antes de cada llamada recursiva a quicksort

En el Merge Sort:

- \$a0 = puntero al inicio del arreglo
- \$a1 = puntero al final del arreglo
- \$a2 = puntero al final (solo en merge)
- \$ra se guarda en la pila porque mergesort se llama a sí misma y también llama a merge

3. ¿Cómo afecta el uso de registros frente a memoria en el rendimiento de los algoritmos de ordenamiento implementados?

Los registros son más rápidos que la memoria porque están dentro del procesador. Por eso siempre usamos registros para operaciones y solo accedemos a memoria cuando es necesario (leer/escribir del arreglo).

Quick Sort: Usa registros para el pivote, índices y valores a comparar. Solo accede a memoria para los intercambios (swaps).

Merge Sort: Usa registros para punteros y valores. Accede más a memoria porque copia todo al arreglo auxiliar y de vuelta.

En conclusión, el Quick Sort hace menos movimientos de memoria porque solo intercambia elementos cuando es necesario, mientras que Merge Sort siempre copia todos los elementos al auxiliar y de vuelta, por lo que Quick Sort suele ser más rápido en la práctica.

4. ¿Qué impacto tiene el uso de estructuras de control (bucles anidados, saltos) en la eficiencia de los algoritmos en MIPS32?

Los bucles anidados y saltos afectan al pipeline del procesador. Quick Sort tiene while anidados y muchos saltos condicionales, lo que puede hacerlo menos predecible. Merge Sort usa bucles separados y un flujo más uniforme, por lo que el procesador lo ejecuta mejor, aunque Quick Sort suele hacer menos movimientos de memoria.

5. ¿Cuáles son las diferencias de complejidad computacional entre el algoritmo Quicksort y el algoritmo Mergesort? ¿Qué implicaciones tiene esto para la implementación en un entorno MIPS32?

Complejidad Computacional

- **Quicksort:** Promedio $O(n \log n)$, peor caso $O(n^2)$ cuando el pivote es el más pequeño o el más grande.
- **Mergesort:** Siempre $O(n \log n)$, no importa cómo estén los datos.

En MIPS32

- Quicksort usa menos memoria (solo la pila para recursión) pero puede ser impredecible.
- Mergesort necesita un arreglo auxiliar (más memoria) pero es más estable y predecible.

Para arreglos pequeños o parcialmente ordenados, Quicksort es más rápido. Pero si necesitamos seguridad y tiempo garantizado, Mergesort es mejor porque siempre es $O(n \log n)$.

6. ¿Cuáles son las fases del ciclo de ejecución de instrucciones en la arquitectura MIPS32 (camino de datos)? ¿En qué consisten?

1. **IF (Instruction Fetch)**: Obtiene la instrucción desde la memoria.
2. **ID (Instruction Decode)**: Decodifica la instrucción y lee los registros.
3. **EX (Execute)**: Ejecuta la operación (ALU: sumas, restas, comparaciones, etc.).
4. **MEM (Memory Access)**: Accede a memoria si la instrucción lo necesita (lw/sw).
5. **WB (Write Back)**: Escribe el resultado en el registro destino.

7. ¿Qué tipo de instrucciones se usaron predominantemente en la práctica (R, I, J) y por qué?

En ambos algoritmos predominan las instrucciones tipo I porque son las más útiles para trabajar con arreglos:

- `lw` y `sw` para leer y escribir elementos del arreglo.
- `addi` para avanzar punteros e índices.
- `beq`, `bne`, `bgt`, `bge`, `ble` para todos los saltos condicionales (bucles y comparaciones).

También usamos instrucciones tipo R como `add`, `sub`, `sll` para cálculos de direcciones (multiplicar índices por 4) y operaciones aritméticas básicas.

Las instrucciones tipo J solo las usamos para `j` (saltos incondicionales en bucles) y `jal/jr` para llamadas y retornos de funciones.

8. ¿Cómo se ve afectado el rendimiento si se abusa del uso de instrucciones de salto (`j`, `beq`, `bne`) en lugar de usar estructuras lineales?

En nuestros códigos:

- Quick Sort tiene más saltos por sus bucles anidados y comparaciones constantes, lo que puede causar más pérdidas en el pipeline.

- Merge Sort tiene menos saltos condicionales y un flujo más secuencial, lo que da mejor rendimiento en el pipeline.

Por eso, aunque Quick Sort suele hacer menos movimientos de memoria, sus muchos saltos pueden reducir su ventaja en MIPS32.

9. ¿Qué ventajas ofrece el modelo RISC de MIPS en la implementación de algoritmos básicos como los de ordenamiento?

- Instrucciones simples que ejecutan en 1 ciclo.
- 32 registros para mantener pivotes, índices y valores sin ir a memoria.
- Control explícito de memoria con `lw/sw`.
- Pipelining eficiente por instrucciones de tamaño fijo.
- Saltos condicionales versátiles para bucles y comparaciones.

Todo esto permite implementar algoritmos de ordenamiento rápidos y eficientes con control total del flujo y la memoria.

10. ¿Cómo se usó el modo de ejecución paso a paso (Step, Step Into) en MARS para verificar la correcta ejecución del algoritmo?

El modo paso a paso nos permitió verificar la ejecución correcta de los algoritmos:

- **Quick Sort:** Revisamos que los índices `i` y `j` avanzaran bien, que el pivote se eligiera correctamente y que los intercambios se hicieran en las posiciones adecuadas, además de comprobar las llamadas recursivas.
- **Merge Sort:** Validamos que la división en mitades funcionara, que el arreglo auxiliar guardara los elementos ordenados y que se copiaran de vuelta sin errores, verificando los punteros en cada bucle.

Así detectamos errores como usar registros temporales cuando debían ser guardados, o no restaurar correctamente los valores en la pila.

11. ¿Qué herramienta de MARS fue más útil para observar el contenido de los registros y detectar errores lógicos?

La ventana de registros (Registers) fue la más útil, ya que nos permitía observar en tiempo real los valores de índices, punteros, pivote y comparaciones, además de verificar que `$ra` y los registros guardados se restauraran correctamente. También usamos la ventana de memoria para confirmar los accesos al arreglo y la pila, pero la de registros fue la principal para detectar errores lógicos.

12. ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo R? (por ejemplo: add)

Se encuentra en el menú **Tools** → **MIPS X-Ray**. Al conectarlo y ejecutar paso a paso, se anima un diagrama de flujo de datos que muestra el camino que sigue la instrucción `add $t1, $t2, $t3`.

- **Instruction Fetch:** Se ilumina el camino desde la Instruction Memory para obtener el código de la instrucción.
- **Decode:** Se ilumina el camino hacia el Register Bank para leer `$t2` y `$t3`.
- **Execute:** La señal viaja a la ALU, que realiza la suma. Se puede ver la ALU Control activando la operación correcta.
- **Memory Access:** Para `add`, este paso está inactivo y no se ilumina en el diagrama (a diferencia de `lw`, que sí ilumina la Data Memory).
- **Write Back:** Se ilumina el camino de vuelta al Register Bank para escribir el resultado en `$t1`.

13. ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo I? (por ejemplo: lw)

En MARS, con el plugin MIPS X-Ray (Tools → MIPS X-Ray), podemos ver el camino de datos de `lw`:

Ejemplo: `lw $t0, 4($s1)`

1. **IF (Instruction Fetch):** Obtiene la instrucción `lw` desde la memoria de instrucciones.
2. **ID (Instruction Decode):** Lee `$s1` y calcula la dirección sumando el offset (4).
3. **EX (Execute):** La ALU suma `$s1 + 4` para calcular la dirección de memoria.
4. **MEM (Memory Access):** La Data Memory se activa y lee el valor en la dirección calculada.
5. **WB (Write Back):** Escribe el valor leído de memoria en `$t0`.

En X-Ray vemos cómo se activa la Data Memory (a diferencia de `add`), el camino pasa por la memoria RAM y luego vuelve al banco de registros.

14. Justificar la elección del algoritmo alternativo

Elegimos Merge Sort porque aunque Quick Sort suele ser más rápido en promedio, Merge Sort siempre es $O(n \log n)$ y no tiene un caso malo como Quick Sort que puede ser $O(n^2)$. En MIPS32 esto es importante porque Merge Sort tiene menos saltos anidados y la ejecución es más uniforme, lo que le sienta mejor al pipeline. También nos permite ver otra forma de ordenar usando un arreglo auxiliar, lo que complementa lo que ya vimos con Quick Sort y nos da una visión más completa de cómo funcionan los algoritmos en ensamblador.

15. Análisis y Discusión de los Resultados

Al comparar Quick Sort y Merge Sort en MIPS32, observamos diferencias significativas en su comportamiento:

- **Rendimiento:** Quick Sort fue más rápido en promedio por hacer menos movimientos de memoria, pero Merge Sort fue más consistente al ser siempre $O(n \log n)$.
- **Uso de memoria:** Quick Sort usa solo la pila (recursión), mientras que Merge Sort necesita un arreglo auxiliar, duplicando el espacio.
- **Implementación:** Quick Sort fue más sencillo (intercambios con índices). Merge Sort fue más complejo por el manejo del arreglo auxiliar y punteros.
- **En MARS:** Quick Sort sufrió más por sus bucles anidados y saltos, mientras que Merge Sort tuvo mejor comportamiento en el pipeline al tener un flujo más uniforme.

Conclusión

Quick Sort es mejor cuando importa la velocidad y la memoria es limitada. Merge Sort es preferible cuando se necesita tiempo garantizado y estabilidad, aunque use más memoria. La elección depende de los requisitos: velocidad vs consistencia.